

МИНИСТЕРСТВО ОБРАЗОВАНИЯ И НАУКИ РФ

Федеральное государственное автономное
образовательное учреждение высшего образования
«Национальный исследовательский университет ИТМО»

ФАКУЛЬТЕТ ПРОГРАММНОЙ ИНЖЕНЕРИИ И КОМПЬЮТЕРНОЙ ТЕХНИКИ

ЛАБОРАТОРНАЯ РАБОТА №4 по дисциплине
‘ВЫЧИСЛИТЕЛЬНАЯ МАТЕМАТИКА’

Вариант №5

Выполнил:
Студент группы Р3210
Давидюк А.Ю.

Преподаватель:
Рыбаков С.Д.

Цель работы: найти функцию, являющуюся наилучшим приближением заданной табличной функции по методу наименьших квадратов.

Ход работы

Аппроксимация функции методом наименьших квадратов

Вариант 5

Исходные данные

$$Y = \frac{6x}{x^4 + 5}, x \in [0; 2], h = 0.2, n = 11$$

Таблица экспериментальных данных

i	1	2	3	4	5	6	7	8	9	10	11
x_i	0.0	0.2	0.4	0.6	0.8	1.0	1.2	1.4	1.6	1.8	2.0
y_i	0.0000	0.2399	0.4776	0.7018	0.8873	1.0000	1.0179	0.9501	0.8309	0.6969	0.5714

1. Линейная аппроксимация: $\varphi(x) = a + bx$

Вычисление сумм:

$$s_x = \sum x_i = 11.0000, s_{xx} = \sum x_i^2 = 15.4000$$
$$s_y = \sum y_i = 7.3737, s_{xy} = \sum x_i y_i = 8.6482$$

Система нормальных уравнений:

$$\begin{cases} n \cdot a + s_x \cdot b = s_y \\ s_x \cdot a + s_{xx} \cdot b = s_{xy} \end{cases} \Rightarrow \begin{cases} 11a + 11b = 7.3737 \\ 11a + 15.4b = 8.6482 \end{cases}$$

Решение:

$$a = 0.3807, b = 0.2896$$

Итоговая формула:

$$\varphi_{\text{lin}}(x) = 0.3807 + 0.2896x$$

Среднеквадратичное отклонение:

$$\sigma_{\text{lin}} = \sqrt{\frac{\sum (\varphi(x_i) - y_i)^2}{n}} = 0.2513$$

2. Квадратичная аппроксимация: $\varphi(x) = a + bx + cx^2$

Дополнительные суммы:

$$s_{xxx} = \sum x_i^3 = 24.2000, s_{xxxx} = \sum x_i^4 = 40.5328$$
$$s_{xxy} = \sum x_i^2 y_i = 11.9051$$

Система нормальных уравнений:

$$\begin{cases} 11a + 11b + 15.4c = 7.3737 \\ 11a + 15.4b + 24.2c = 8.6482 \\ 15.4a + 24.2b + 40.5328c = 11.9051 \end{cases}$$

Решение (методом Крамера/Гаусса):

$$a = -0.0419, b = 1.6984, c = -0.7044$$

Итоговая формула:

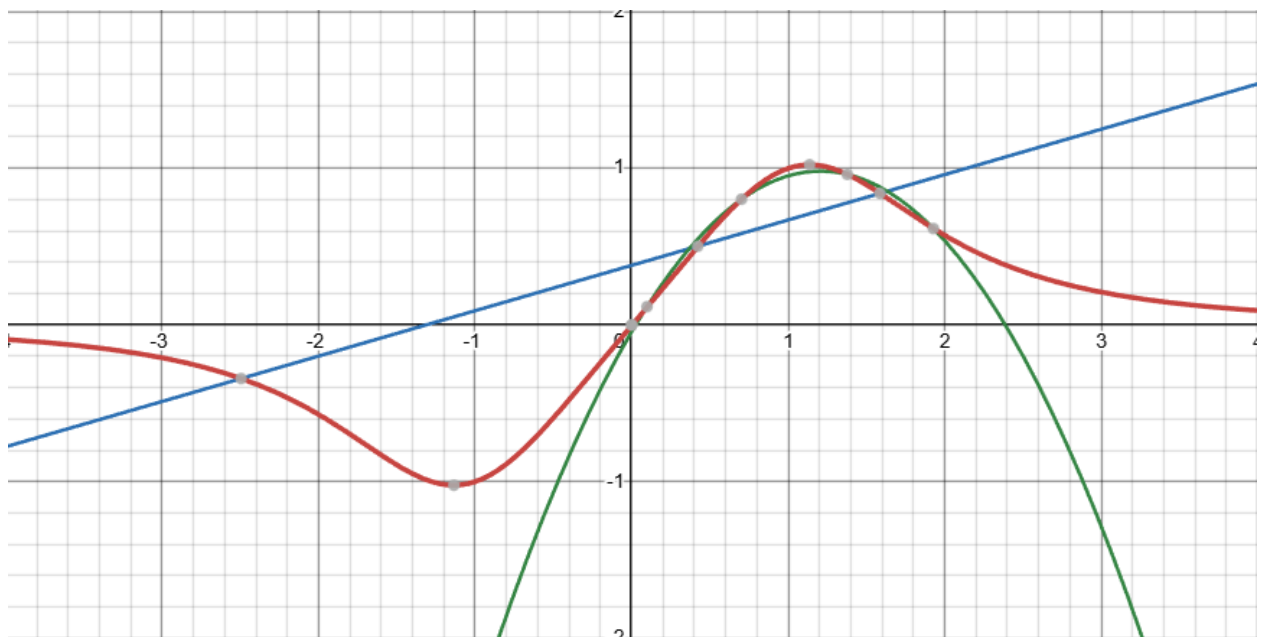
$$\varphi_{\text{quad}}(x) = -0.0419 + 1.6984x - 0.7044x^2$$

Среднеквадратичное отклонение:

$$\sigma_{\text{quad}} = \sqrt{\frac{\sum (\varphi(x_i) - y_i)^2}{n}} = 0.0351$$

3. Сравнение и вывод

Метод	Формула	σ
Линейная	$0.3807 + 0.2896x$	0.2513
Квадратичная	$-0.0419 + 1.6984x - 0.7044x^2$	0.0351



2. Программная реализация задачи

```

2. Ввести данные вручную
3. Сгенерировать тестовые данные (функция  $y = 6x/(x^2+5)$ )
4. Выход
Ваш выбор (1-4): 1
Путь к файлу [data.txt]: l4\data.txt
Загружено 11 точек из l4\data.txt
РЕЗУЛЬТАТЫ АППРОКСИМАЦИИ МЕТОДОМ НАИМЕНЬШИХ КВАДРАТОВ
НАИЛУЧШАЯ МОДЕЛЬ: CUBIC

```

Модель	Формула	σ (СКО)	R^2	Качество
linear	$y = 0.2896x + 0.3807$	0.2513	0.3470	Плохая аппроксимация ($r=0.5890$)
quadratic	$y = -0.7044x^2 + 1.6984x - 0.0419$	0.0351	0.9872	Отличная аппроксимация
cubic	$y = -0.0221x^3 + \dots - 0.0356$	0.0349	0.9874	Отличная аппроксимация
exponential	Пропущено: требуется $y > 0$ для всех точек.			
logarithmic	Пропущено: требуется $x > 0$ для всех точек.			
power	Пропущено: требуется $x > 0$ и $y > 0$.			

ДЕТАЛЬНАЯ ТАБЛИЦА ДЛЯ CUBIC МОДЕЛИ:

№	x_i	y_i	$\phi(x_i)$	$\epsilon_i = \phi - y$
1	0.0000	0.0000	-0.0356	-0.0356
2	0.2000	0.2399	0.2683	0.0284
3	0.4000	0.4776	0.5201	0.0425
4	0.6000	0.7018	0.7187	0.0169
5	0.8000	0.8873	0.8630	-0.0243
6	1.0000	1.0000	0.9521	-0.0479
7	1.2000	1.0179	0.9848	-0.0331
8	1.4000	0.9501	0.9601	0.0100
9	1.6000	0.8309	0.8770	0.0461
10	1.8000	0.6969	0.7343	0.0374
11	2.0000	0.5714	0.5310	-0.0404

```

import numpy as np
import matplotlib.pyplot as plt
import os
import sys
import warnings
from typing import Tuple, Dict, Any, Optional

warnings.filterwarnings('ignore')

def load_table_from_file(filepath: str) -> Tuple[np.ndarray, np.ndarray]:
    if not os.path.exists(filepath):
        raise FileNotFoundError(f"📁 Файл '{filepath}' не найден.")

    x_vals, y_vals = [], []
    try:
        with open(filepath, 'r', encoding='utf-8') as f:
            for line_num, line in enumerate(f, 1):
                line = line.strip().replace(',', '.')
                if not line or line.startswith('#') or
line.lower().startswith(('x', '№', 'index', 'x_')):
                    continue

                parts = line.split()
                if len(parts) < 2:
                    continue

                try:
                    x_vals.append(float(parts[0]))
                    y_vals.append(float(parts[1]))
                except ValueError:
                    continue
    except Exception as e:
        raise RuntimeError(f"❌ Ошибка чтения файла: {e}")

    if not x_vals:
        raise ValueError(" В файле не найдено валидных пар чисел (x y).")
    if not (8 <= len(x_vals) <= 12):
        raise ValueError(f" По заданию требуется 8-12 точек. В файле найдено: {len(x_vals)}.")

    return np.array(x_vals), np.array(y_vals)

```

2. КЛАСС АППРОКСИМАЦИИ (МНК)

```
class LSApproximator:
```

```
    def __init__(self, x: np.ndarray, y: np.ndarray):
```

```

self.x = np.array(x, dtype=float)
self.y = np.array(y, dtype=float)
self.n = len(x)
self.results: Dict[str, Any] = {}

def _mse(self, y_pred: np.ndarray) -> float:
    return np.sqrt(np.mean((y_pred - self.y)**2))

def _r2(self, y_pred: np.ndarray) -> float:
    ss_res = np.sum((self.y - y_pred)**2)
    ss_tot = np.sum((self.y - np.mean(self.y))**2)
    return 1.0 if ss_tot < 1e-12 else 1 - ss_res / ss_tot

def _pearson(self, y_pred: np.ndarray) -> float:
    num = np.sum((self.x - np.mean(self.x)) * (self.y - np.mean(self.y)))
    den = np.sqrt(np.sum((self.x - np.mean(self.x))**2) * np.sum((self.y -
np.mean(self.y))**2))
    return 0.0 if den < 1e-12 else num / den

def _register(self, name: str, func, params: dict, y_pred: np.ndarray, err:
Optional[str] = None):
    self.results[name] = {
        'func': func, 'params': params, 'y_pred': y_pred,
        'mse': self._mse(y_pred), 'r2': self._r2(y_pred),
        'error': err
    }
    if name == 'linear':
        self.results[name]['pearson'] = self._pearson(y_pred)

def fit_all(self):
    # 1. Линейная:  $y = ax + b$ 
    try:
        c = np.polyfit(self.x, self.y, 1)
        self._register('linear', lambda x: c[0]*x + c[1], {'a': c[0], 'b':
c[1]}, np.polyval(c, self.x))
    except Exception as e: self.results['linear'] = {'error': f" {e}"}

    # 2. Квадратичная:  $y = ax^2 + bx + c$ 
    try:
        c = np.polyfit(self.x, self.y, 2)
        self._register('quadratic', lambda x: c[0]*x**2 + c[1]*x + c[2],
{'a': c[0], 'b': c[1], 'c': c[2]}, np.polyval(c, self.x))
    except Exception as e: self.results['quadratic'] = {'error': f" {e}"}

    # 3. Кубическая:  $y = ax^3 + bx^2 + cx + d$ 
    try:
        c = np.polyfit(self.x, self.y, 3)
        self._register('cubic', lambda x: c[0]*x**3 + c[1]*x**2 + c[2]*x +
c[3], {'a': c[0], 'b': c[1], 'c': c[2], 'd': c[3]}, np.polyval(c, self.x))
    except Exception as e: self.results['cubic'] = {'error': f" {e}"}

```

```

# 4. Экспоненциальная
if np.all(self.y > 0):
    try:
        c = np.polyfit(self.x, np.log(self.y), 1)
        b, ln_a = c
        a = np.exp(ln_a)
        y_p = a * np.exp(b * self.x)
        self._register('exponential', lambda x, a=a, b=b: a*np.exp(b*x),
{'a': a, 'b': b}, y_p)
    except Exception as e: self.results['exponential'] = {'error': f"
{e}"}
    else:
        self.results['exponential'] = {'error': " Пропущено: требуется y > 0
для всех точек."}

# 5. Логарифмическая
if np.all(self.x > 0):
    try:
        c = np.polyfit(np.log(self.x), self.y, 1)
        a, b = c
        y_p = a * np.log(self.x) + b
        self._register('logarithmic', lambda x, a=a, b=b: a*np.log(x)+b,
{'a': a, 'b': b}, y_p)
    except Exception as e: self.results['logarithmic'] = {'error': f"
{e}"}
    else:
        self.results['logarithmic'] = {'error': " Пропущено: требуется x > 0
для всех точек."}

# 6. Степенная
if np.all(self.x > 0) and np.all(self.y > 0):
    try:
        c = np.polyfit(np.log(self.x), np.log(self.y), 1)
        b, ln_a = c
        a = np.exp(ln_a)
        y_p = a * self.x**b
        self._register('power', lambda x, a=a, b=b: a*(x**b), {'a': a,
'b': b}, y_p)
    except Exception as e: self.results['power'] = {'error': f" {e}"}
    else:
        self.results['power'] = {'error': " Пропущено: требуется x > 0 и y >
0."}

def get_best(self) -> Tuple[str, Any]:
    valid = {k: v for k, v in self.results.items() if v.get('error') is None}
    if not valid: return "Нет успешных моделей", {}
    best = min(valid, key=lambda k: valid[k]['mse'])
    return best, valid[best]

@staticmethod
def quality_msg(r2: float) -> str:

```

```

if r2 >= 0.95: return " Отличная аппроксимация"
elif r2 >= 0.85: return " Хорошая аппроксимация"
elif r2 >= 0.70: return " Удовлетворительная аппроксимация"
else: return " Плохая аппроксимация"

```

3. ВЫВОД И ВИЗУАЛИЗАЦИЯ

```

def print_report(approx: LSApproximator):

    print(" РЕЗУЛЬТАТЫ АППРОКСИМАЦИИ МЕТОДОМ НАИМЕНЬШИХ КВАДРАТОВ")

    best_name, best_res = approx.get_best()
    print(f" НАИЛУЧШАЯ МОДЕЛЬ: {best_name.upper()}")

    # Сводная таблица по всем моделям
    print(f"\n{'Модель':<14} {'Формула':<35} {'σ (СКО)':<10} {'R²':<8} {'Качество'}")
    print("-"*80)
    for name, res in approx.results.items():
        if res.get('error'):
            print(f"{name:<14} {res['error']:<70}")
            continue
        r2 = res['r2']
        mse = res['mse']
        form = res.get('formula', 'N/A')

        if name == 'linear': form = f"y = {res['params']['a']:.4f}x + {res['params']['b']:.4f}"
        elif name == 'quadratic': form = f"y = {res['params']['a']:.4f}x² + {res['params']['b']:.4f}x + {res['params']['c']:.4f}"
        elif name == 'cubic': form = f"y = {res['params']['a']:.4f}x³ + ... + {res['params']['d']:.4f}"
        elif name == 'exponential': form = f"y = {res['params']['a']:.4f} * e^{(res['params']['b']:.4f)x}"
        elif name == 'logarithmic': form = f"y = {res['params']['a']:.4f} * ln(x) + {res['params']['b']:.4f}"
        elif name == 'power': form = f"y = {res['params']['a']:.4f} * x^{res['params']['b']:.4f}"

        qual = approx.quality_msg(r2)
        pearson_str = f" (r={res['pearson']:.4f})" if name == 'linear' else ""
        print(f"{name:<14} {form:<35} {mse:<10.4f} {r2:<8.4f} {qual}{pearson_str}")
    print("-"*80)

    # Детальная таблица для лучшей модели (xi, yi, φ(xi), εi)
    if best_res and 'y_pred' in best_res:
        print(f"\n ДЕТАЛЬНАЯ ТАБЛИЦА ДЛЯ {best_name.upper()} МОДЕЛИ:")

```

```

        print(f"{'N':<4} {'x_i':<10} {'y_i':<10} {' $\phi(x_i)$ ':<12} {' $\epsilon_i = \phi-$   

y':<12}")
    print("-"*50)
    for i in range(approx.n):
        eps = best_res['y_pred'][i] - approx.y[i]
        print(f"{i+1:<4} {approx.x[i]:<10.4f} {approx.y[i]:<10.4f}  

{best_res['y_pred'][i]:<12.4f} {eps:<12.4f}")
    print("-"*50)

def save_to_file(approx: LSApproximator, filename: str):
    with open(filename, 'w', encoding='utf-8') as f:
        f.write("РЕЗУЛЬТАТЫ АППРОКСИМАЦИИ МНК\n")
        f.write("="*60 + "\n")
        best_name, best_res = approx.get_best()
        f.write(f"Лучшая модель: {best_name}\n\n")

        for name, res in approx.results.items():
            f.write(f"[{name.upper()}]\n")
            if res.get('error'):
                f.write(f"Статус: {res['error']}\n\n")
                continue
            f.write(f"Параметры: {res['params']}\n")
            f.write(f"CKO ( $\sigma$ ): {res['mse']:.6f}\n")
            f.write(f"R2: {res['r2']:.6f}\n")
            if name == 'linear': f.write(f"Коэф. корреляции (r):  

{res['pearson']:.6f}\n")
            f.write(f"Качество: {approx.quality_msg(res['r2'])}\n")

            f.write("Данные (x, y,  $\phi(x)$ ,  $\epsilon$ ): \n")
            for i in range(approx.n):
                eps = res['y_pred'][i] - approx.y[i]
                f.write(f"{approx.x[i]:.4f} | {approx.y[i]:.4f} |  

{res['y_pred'][i]:.4f} | {eps:.4f}\n")
            f.write("\n")
        print(f" Отчёт сохранён в: {os.path.abspath(filename)}")

def plot_all(approx: LSApproximator, title: str = "Аппроксимация МНК"):
    plt.figure(figsize=(10, 6))
    plt.scatter(approx.x, approx.y, color='black', s=60, zorder=5,  

label='Исходные точки')

    x_dense = np.linspace(np.min(approx.x) - 0.3, np.max(approx.x) + 0.3, 500)
    colors = {'linear': 'blue', 'quadratic': 'green', 'cubic': 'purple',  

'exponential': 'orange', 'logarithmic': 'red', 'power': 'brown'}
    styles = {'linear': '-', 'quadratic': '--', 'cubic': ':',  

'exponential': '-.', 'logarithmic': '--', 'power': '-.'}

    valid_models = {k: v for k, v in approx.results.items() if v.get('error') is  

None}
    for name, res in valid_models.items():
        try:

```

```

        y_plot = res['func'](x_dense)
        plt.plot(x_dense, y_plot, color=colors.get(name, 'gray'),
linestyle=styles.get(name, '-'),
                    linewidth=1.5, alpha=0.7, label=f"{name.title()}
(σ={res['mse']:.3f})")
    except: pass

plt.xlabel('x', fontsize=12); plt.ylabel('y', fontsize=12)
plt.title(title, fontsize=14, fontweight='bold')
plt.legend(fontsize=9)
plt.grid(True, alpha=0.3)

all_y = [approx.y] + [res['func'](x_dense) for res in valid_models.values()]
if 'func' in res:
    flat_y = np.concatenate(all_y)
    flat_y = flat_y[np.isfinite(flat_y)]
    if len(flat_y) > 0:
        margin = 0.15 * (np.max(flat_y) - np.min(flat_y))
        plt.ylim(np.min(flat_y) - margin, np.max(flat_y) + margin)

plt.tight_layout()
plt.show()

```

4. ГЛАВНЫЙ ИНТЕРФЕЙС (КОНСОЛЬ)

```

def safe_input_float(prompt: str, default: Optional[float] = None, required: bool
= True) -> Optional[float]:
    while True:
        try:
            raw = input(prompt).strip()
            if not raw and default is not None: return default
            if not raw and not required: return None
            val = float(raw.replace(',', '.'))
            return val
        except ValueError: print(" Введите корректное число.")
        except KeyboardInterrupt: sys.exit(0)

def main():
    print("\n ЛАБОРАТОРНАЯ РАБОТА №4 | АППРОКСИМАЦИЯ МЕТОДОМ МНК")
    print("="*60)

    while True:
        print("\n Источник данных:")
        print("1. Загрузить таблицу из файла (.txt)")
        print("2. Ввести данные вручную")
        print("3. Сгенерировать тестовые данные (функция  $y = 6x/(x^4+5)$ ")
        print("4. Выход")

        choice = input("Ваш выбор (1-4): ").strip()
        if choice == '4': break

```

```

x, y = None, None
try:
    if choice == '1':
        path = input("Путь к файлу [data.txt]: ").strip() or "data.txt"
        x, y = load_table_from_file(path)
        print(f" Загружено {len(x)} точек из {path}")

    elif choice == '2':
        print(" Введите пары x y (минимум 8, максимум 12). Пустая строка
для завершения.")
        data = []
        while True:
            line = input().strip()
            if not line: break
            parts = line.replace(',', '.').split()
            if len(parts) >= 2:
                try: data.append((float(parts[0]), float(parts[1])))
                except: print("⚠ Игнорирую строку с ошибкой формата.")
            if len(data) < 8: raise ValueError("⚠ Введено менее 8 точек.")
        x = np.array([d[0] for d in data])
        y = np.array([d[1] for d in data])

    elif choice == '3':
        a = safe_input_float("Начало интервала [0]: ", 0.0)
        b = safe_input_float("Конец интервала [2]: ", 2.0)
        h = safe_input_float("Шаг [0.2]: ", 0.2)
        noise = safe_input_float("Уровень шума [0.0]: ", 0.0)

        if b <= a or h <= 0: raise ValueError(" Некорректный интервал или
шаг.")

        x = np.arange(a, b + h/2, h)
        y = (6 * x) / (x**4 + 5) + np.random.normal(0, noise, len(x))
        print(f" Сгенерировано {len(x)} точек")
    else:
        print(" Неверный выбор.")
        continue

# Запуск расчётов
approx = LSApproximator(x, y)
approx.fit_all()

# Вывод
print_report(approx)

# Графики
if input("\n Построить графики? (y/n): ").strip().lower() == 'y':
    plot_all(approx, f"МНК Аппроксимация ({len(x)} точек)")

# Сохранение

```

```

        if input("\n Сохранить отчёт в файл? (y/n): ").strip().lower() ==
'y':
            fname = input("Имя файла [report_l4.txt]: ").strip() or
"report_l4.txt"
            save_to_file(approx, fname)

    except KeyboardInterrupt:
        print("\n👉 Программа остановлена пользователем.")
        break
    except Exception as e:
        print(f"\n Ошибка: {e}")
        print(" Попробуйте другой набор данных или проверьте формат ввода.")

print("\n Работа программы завершена.")

if __name__ == "__main__":
    try:
        main()
    except Exception as e:
        print(f" Критическая ошибка: {e}")
    finally:
        sys.exit(0)

```

Вывод

В ходе выполнения данной лабораторной работы разобрался в способах аппроксимации функций.